# MapReduce and Parallel Data Processing

CA4022: Data (Analytics) at Speed and Scale alessandra.mileo@dcu.ie

# Outline

1. Introduction
    1. Divide and Conquer
    2. Map and reduce
2. MR Execution Phases
    1. Example: Counting Words
3. Optimisations and Fault Tolerance

# Part I

Intro to MR

# How Programmers Solve Problems I

- Divide the problem into parts:
  1. Identical individual pieces
  2. Independent of each other
  3. Able to be completed in any order

- Conquer
  - Solve each part until they're all done

- Combine

# Examples?

- Binary search (of an ordered array)
- Mergesort
- Quicksort
- …

# How Programmers Solve Problems II

- Abstraction
- Decide the parts of the problem that are important, and ignore the rest
- Treat similar things as equivalent

# Data-Level Parallelism

- Data distributed across different nodes that are operating in parallel, can be of two types:
    1. Lots of data in memory, that can be operated on in parallel (e.g. adding 2 – very long – arrays )
    2. Lots of data on many disks that can be operated on in parallel (e.g. Google read-only search!)
- As opposed to Task-level parallelism: different processes (or threads) in parallel on the same or different data

# MapReduce

- Data-parallel programming model
- Pioneered by Google
- Focuses on **scalability**, **cost efficiency** and **fault-tolerance**
- Popularized by open-source project Hadoop (more on this later)

# MapReduce: Simplified Data Processing on Large Clusters

*Jeffrey Dean and Sanjay Ghemawat*

- **Special-purpose computations** that process **large amounts of raw data** (by Google)

- hides the **messy details** of parallelisation in a library (little programming experience required)

- Is there a simple way to abstract the distribution, and to divide the problem?

Why divide?

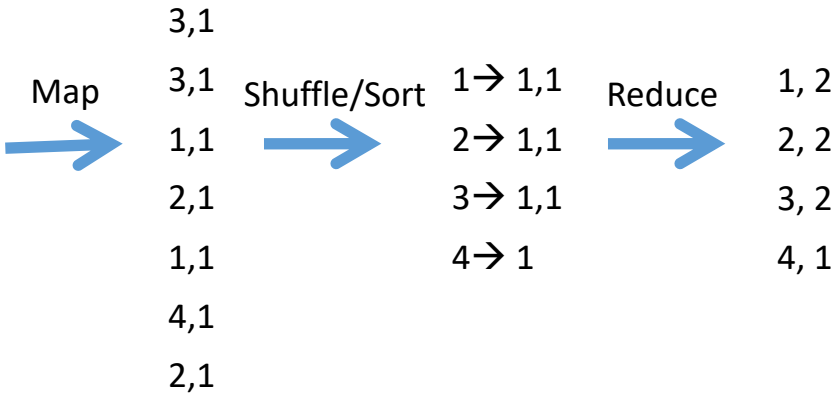
http://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf

# Map and Reduce

- Key/Value pairs → MapReduce → NEW Key/Value pairs
- Map()
  - Split input and run Map on each item in the set *Independent!*
  - emits new key-value pairs
- Reduce()
  - run for each unique key produced by map() *Independent!*
  - Combine results and emits final output

# Example: How many movie ratings ?
(MovieLens)

| USER ID | MOVIE ID | RATING | TIMESTAMP |
|---|---|---|---|
| 112 | 226 | 3 | 891255949 |
| 241 | 303 | 3 | 881253949 |
| 112 | 245 | 1 | 881257434 |
| 367 | 711 | 2 | 876254921 |
| 514 | 226 | 1 | 883974866 |
| 367 | 45 | 4 | 885182433 |
| 112 | 412 | 2 | 887111884 |




See Python code for this on https://www.youtube.com/watch?v=QGgAFG2a7Ic
See more native Java examples at https://www.slideshare.net/andreaiacono/mapreduce-34478449

# MapReduce functions

$$\text{Map}(K_{in}, V_{in}) \rightarrow \text{list}(K_{inter}, V_{inter})$$

$$\text{Reduce}(K_{inter}, \text{list}(V_{inter})) \rightarrow \text{list}(K_{out}, V_{out})$$

# Part II

MR execution and example

# Working Example: Counting words

- We have a collection of text in a file, we read the file line by line
- We want to know how many instances of each particular word are in the text

- What are…
  - …$K_{in}$, $V_{in}$?
  - …$K_{inter}$, $V_{inter}$?
  - …$K_{out}$, $V_{out}$?

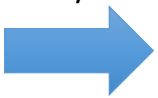
# Wordcount: Pseudocode

```
Map(String document_id, String content):
for each word w in content:
        outputIntermediate(w,1)

(shuffle and sort to bundle the counts for each word)

Reduce(String w, Iterator counts):
        output(w, sum(counts))
```

# Wordcount illustrated

I saw Bob and Lucy.
Bob threw and saw Lucy running.

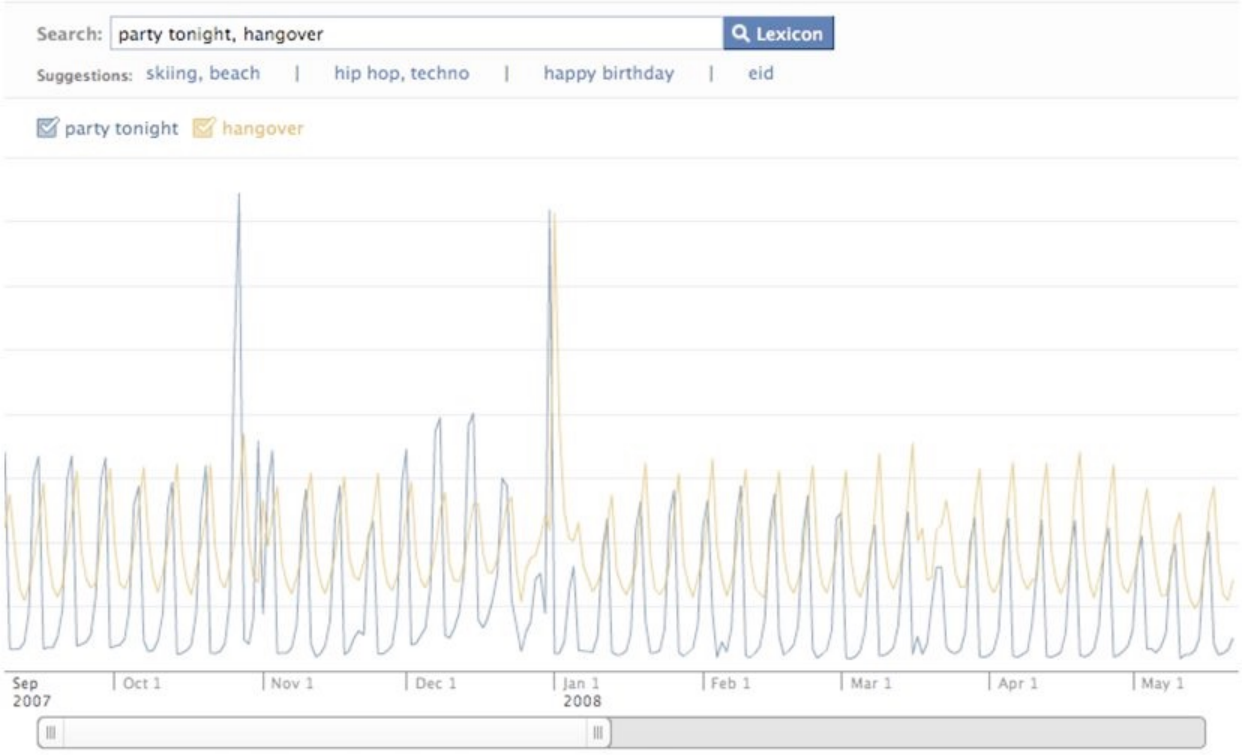

| | |
|---|---|
| I | 1 |
| saw | 1 |
| Bob | 1 |
| and | 1 |
| Lucy | 1 |
| Bob | 1 |
| threw | 1 |
| and | 1 |
| saw | 1 |
| Lucy | 1 |
| running | 1 |




| | |
|---|---|
| and | 1,1 |
| Bob | 1,1 |
| I | 1 |
| Lucy | 1,1 |
| running | 1 |
| saw | 1,1 |
| threw | 1 |


| | |
|---|---|
| and | 2 |
| Bob | 2 |
| I | 1 |
| Lucy | 2 |
| running | 1 |
| saw | 2 |
| threw | 1 |

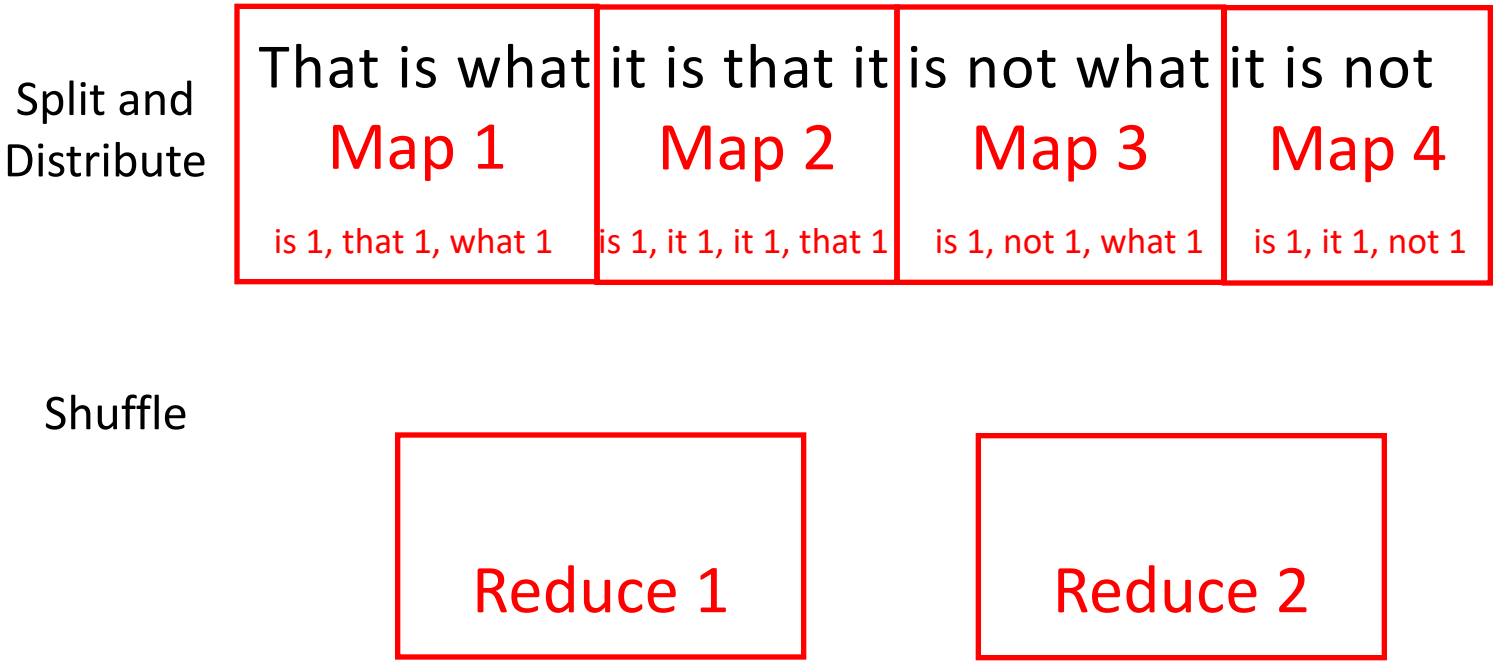
# Example: Facebook Lexicon



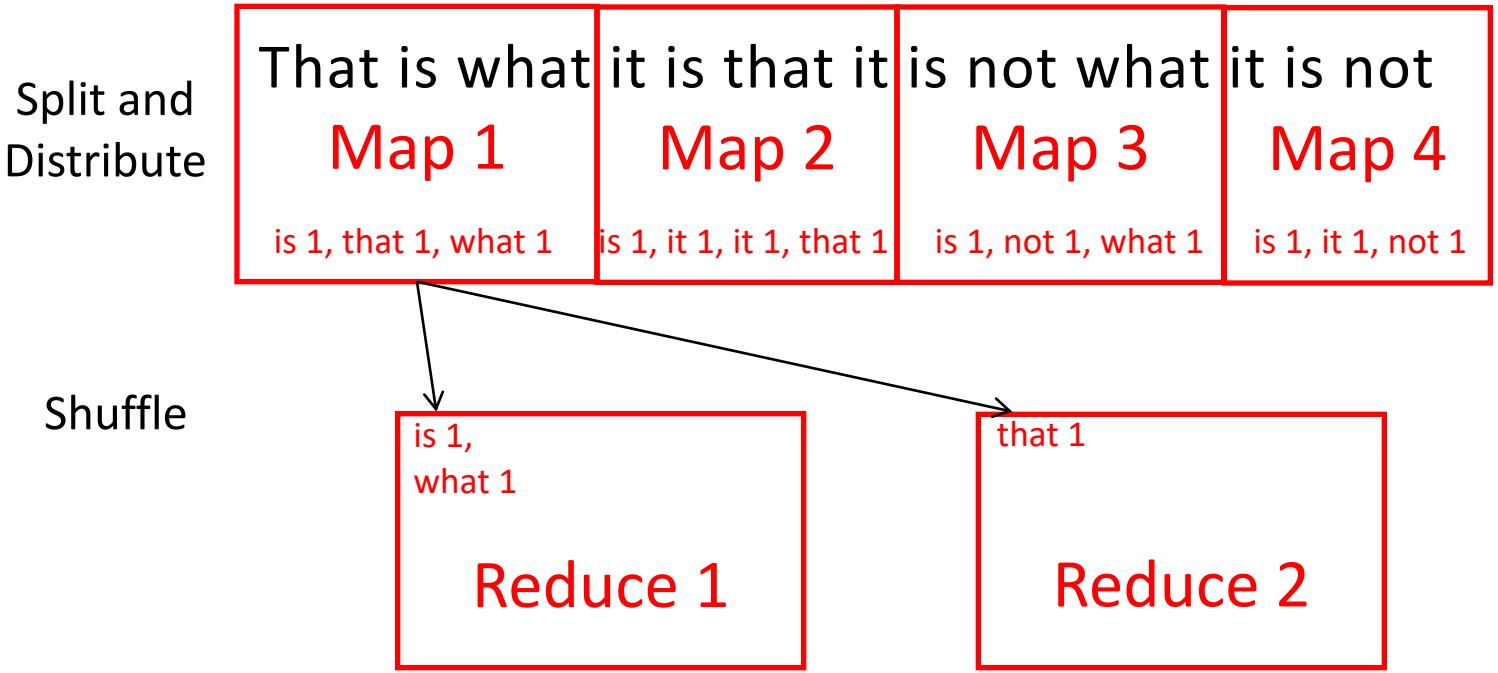

www.facebook.com/lexicon (no longer available)

# MapReduce Phases of Execution

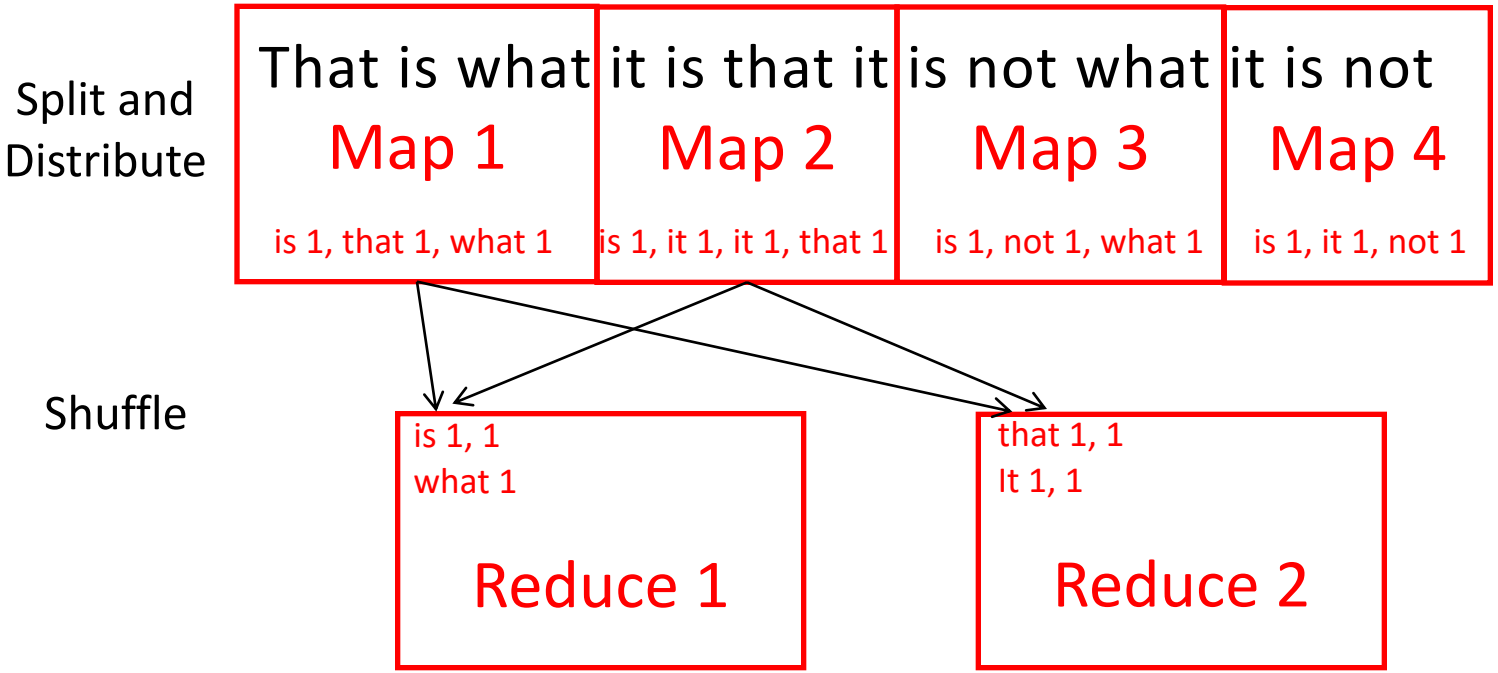
# Wordcount Example



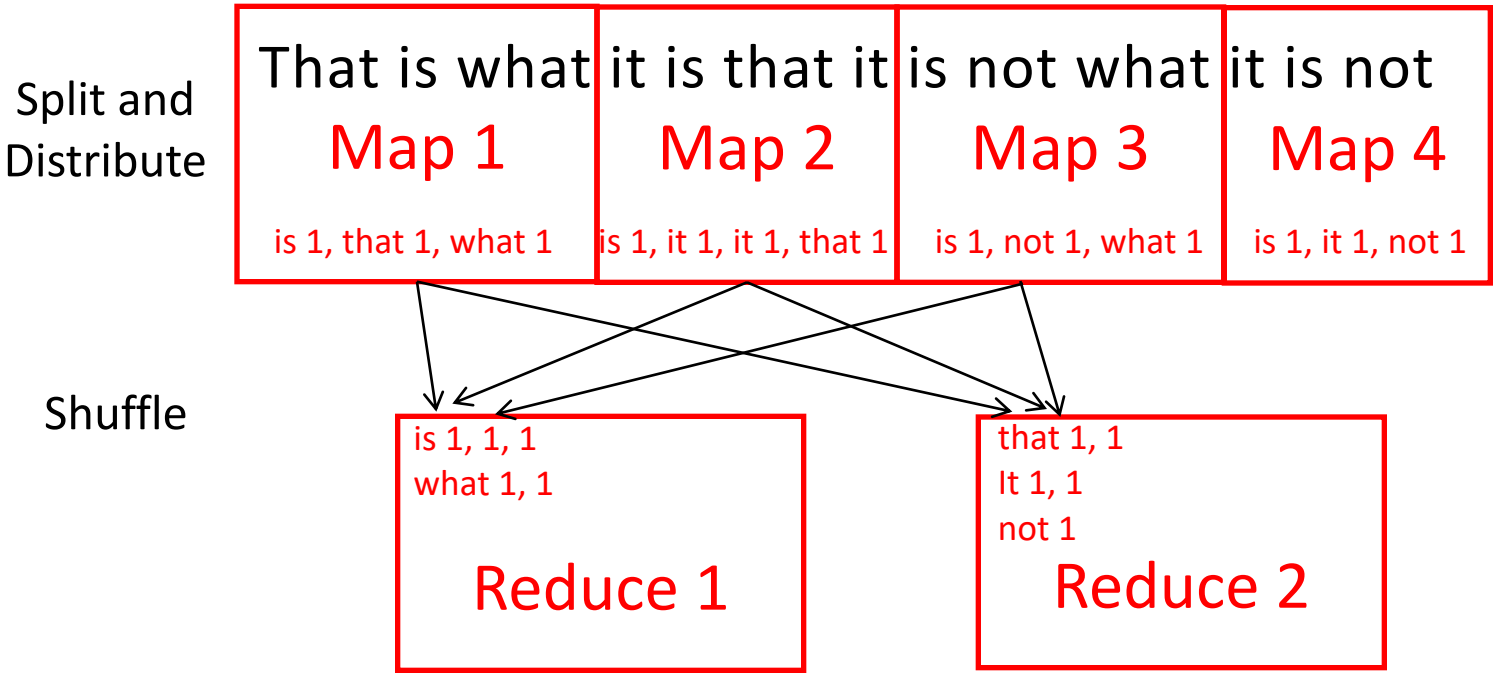
# Wordcount Example



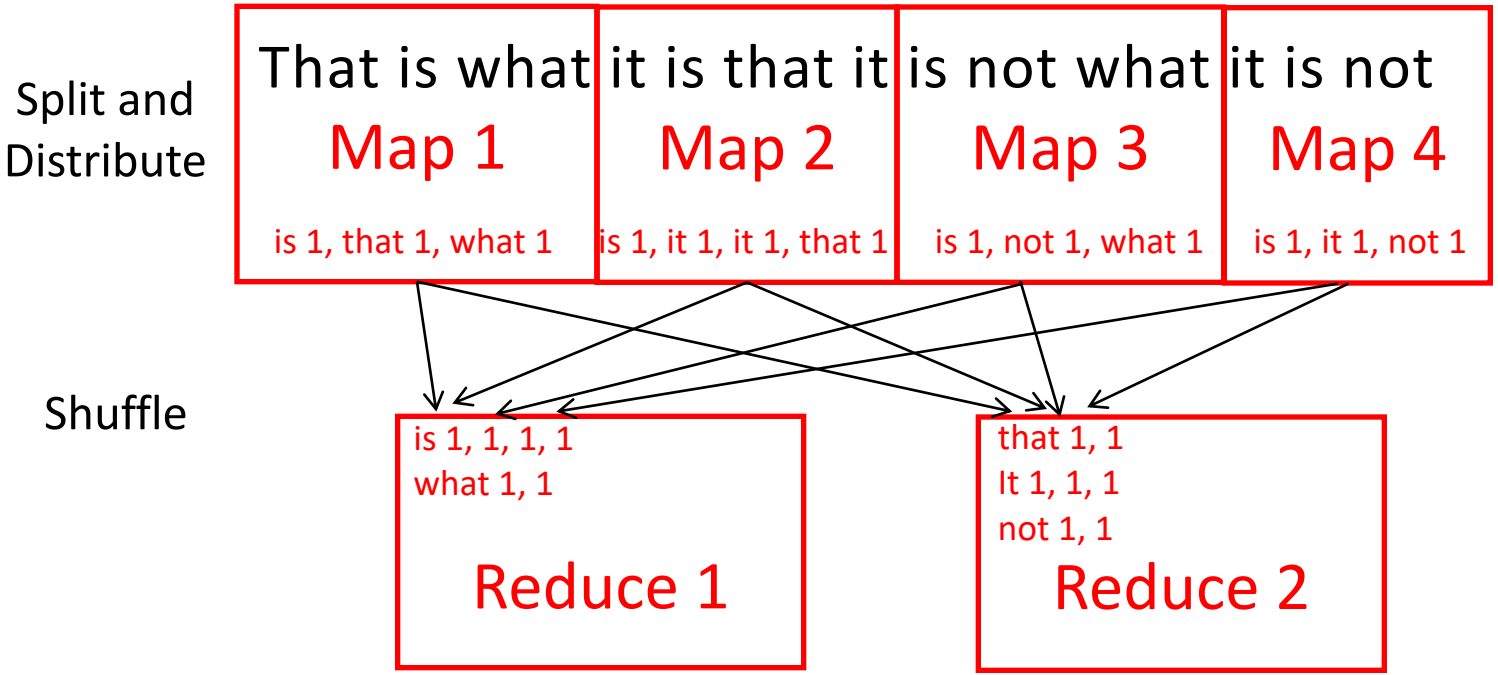
# Wordcount Example



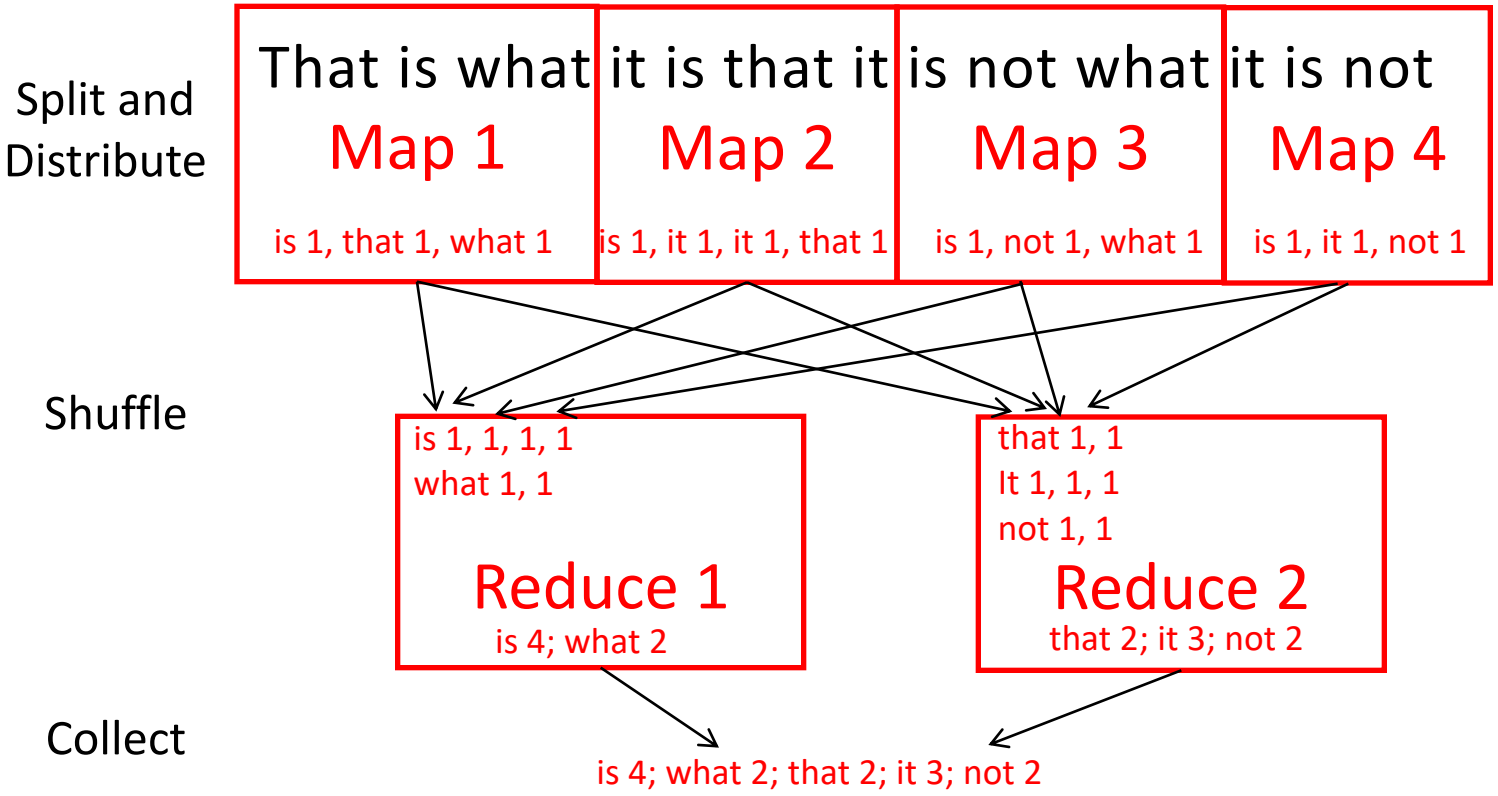
# Wordcount Example



# Wordcount Example



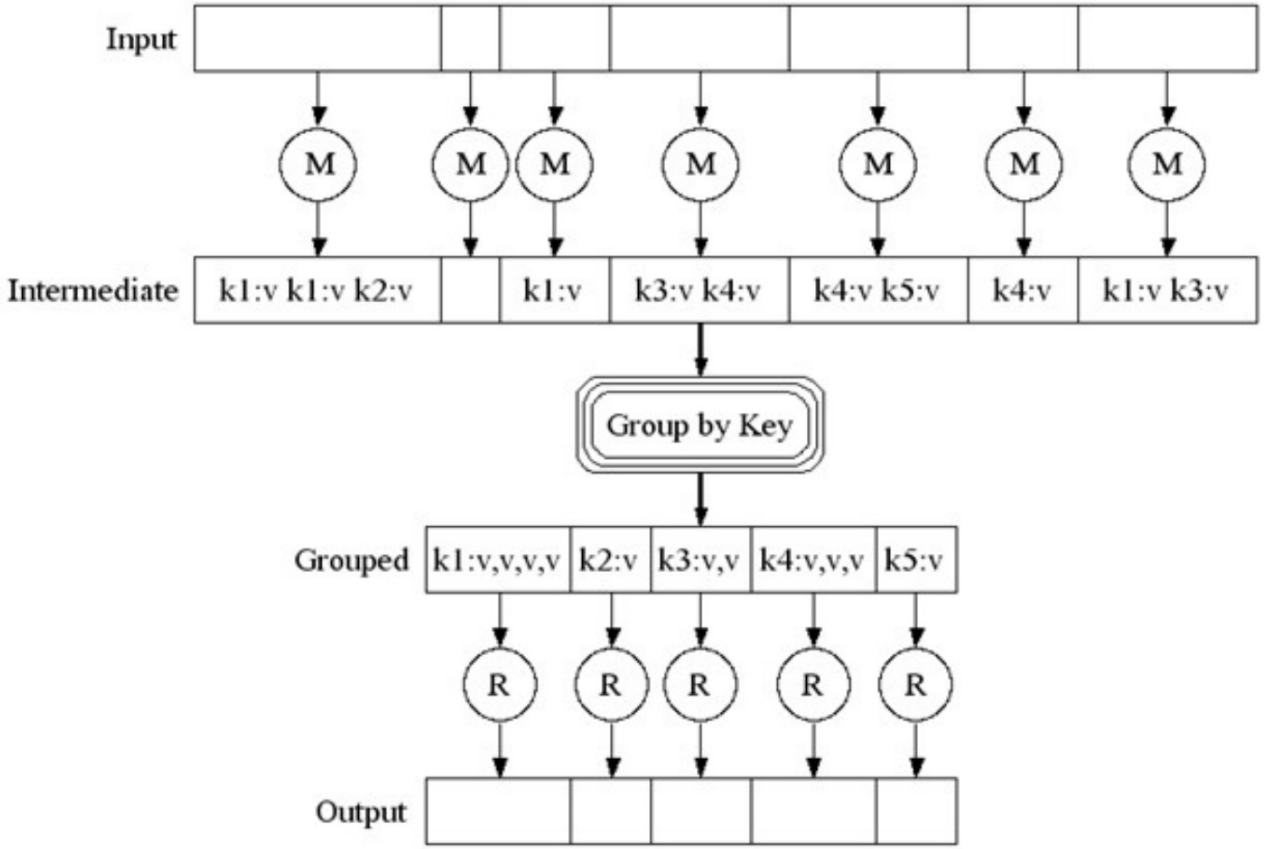
# Wordcount Example



# Part III

Optimisation and Fault Tolerance

# MapReduce Phases of Execution



# MapReduce “Divide and Conquer”

- Map and Reduce operations provided by the user in functional model
- Focus on problem, while MapReduce library deals with messy details
- Framework handles parallelisation
- Fault tolerance handled by re-execution (more later)

# Under the hood

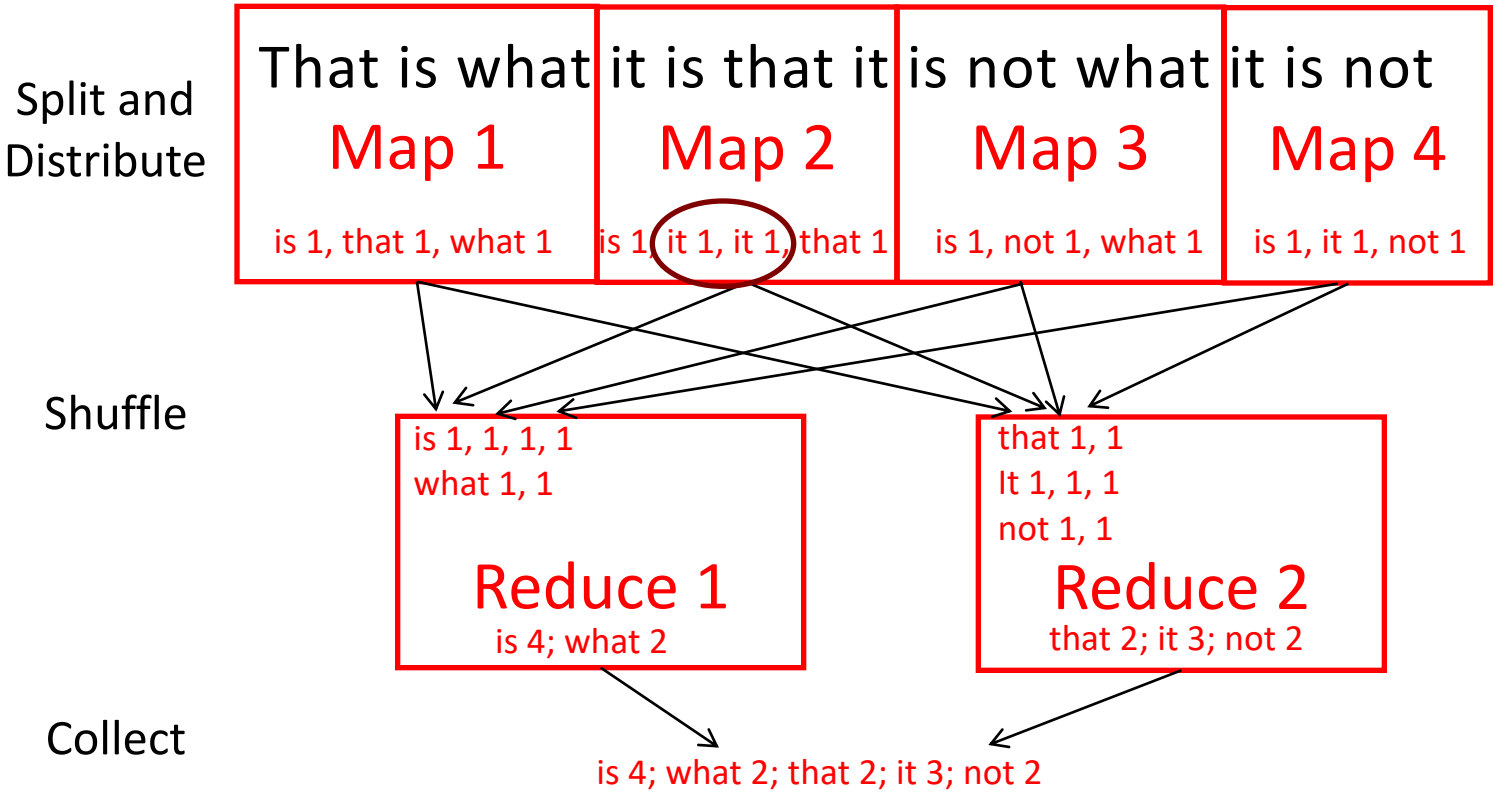
- Execution:
  - Partition key/value into chunks, run map() in parallel
  - After map()s are complete, consolidate values for each unique output key
  - Partition space and run reduce() in parallel
  - If map() or reduce() fails, re-execute!

- Implementation:
  - C++ Library linked to user programs
- Distributed File System and Job Scheduling System (in Hadoop)

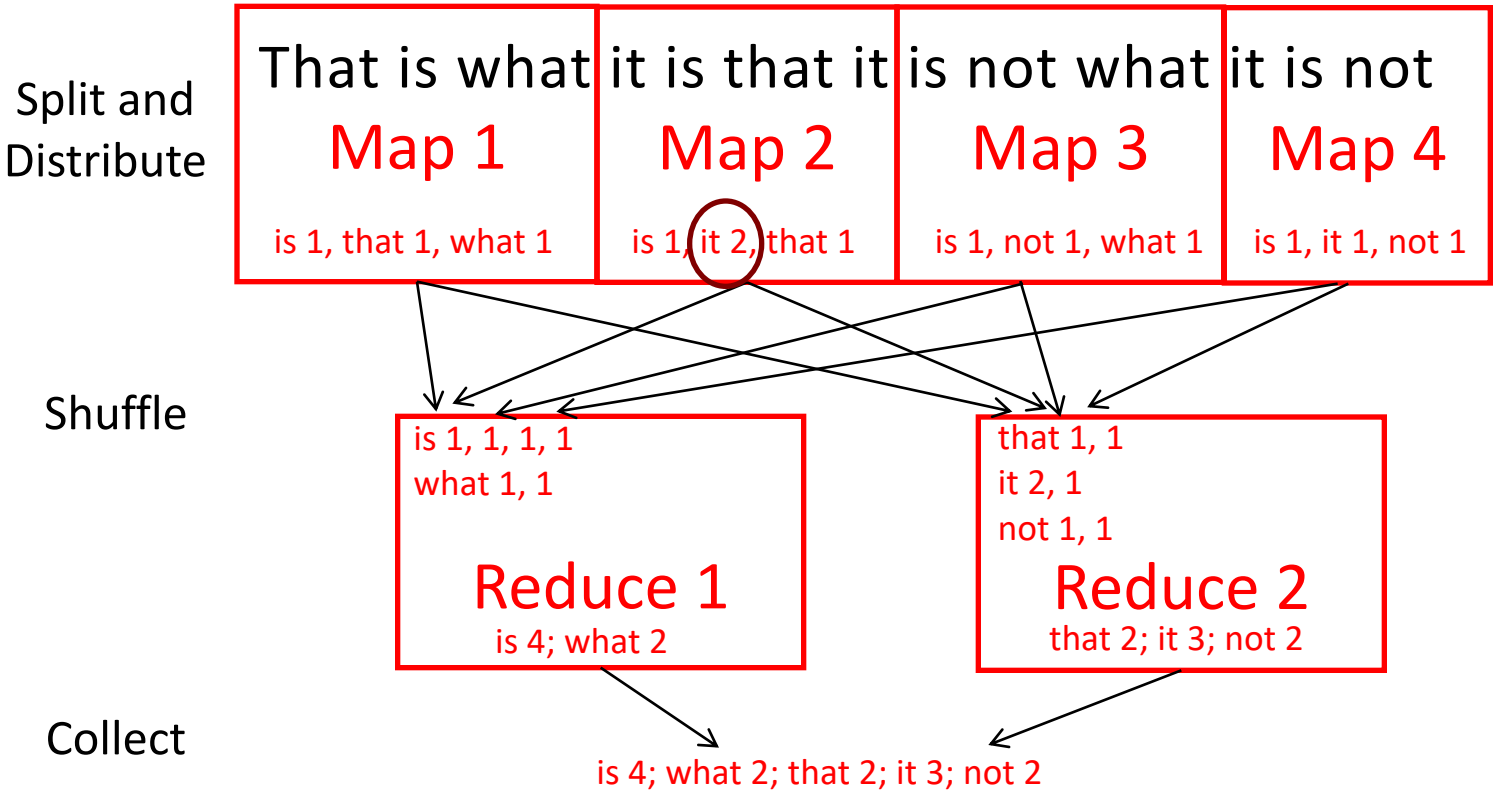
# Optimization: Combiner

- $K_{inter}$, is not always uniformly distributed (e.g. number of "the" in word-count)
- One optimisation is to group ($K_{inter}$, $V_{inter}$ ) before routing them to reducers
- This is achieved with a combine operation that "compresses" the output of a single mapper
- Cannot always be done (need commutative and associative reduce function)

# Wordcount Example (not optimised)



# Wordcount Example (optimised)



# Task Granularity: How many Mappers/Reducers?

- Divide the Map task into M pieces
- Divide the Reduce task into R pieces
- Perform the tasks on W machines

- Master node must:
  - Make O(M+R) scheduling decisions
  - Maintain state for O(M*R) tasks

→ Want M >> W and R > W
(e.g. M = 200,000; R = 6,000; W = 2,000)

# Mapreduce Phases of Execution

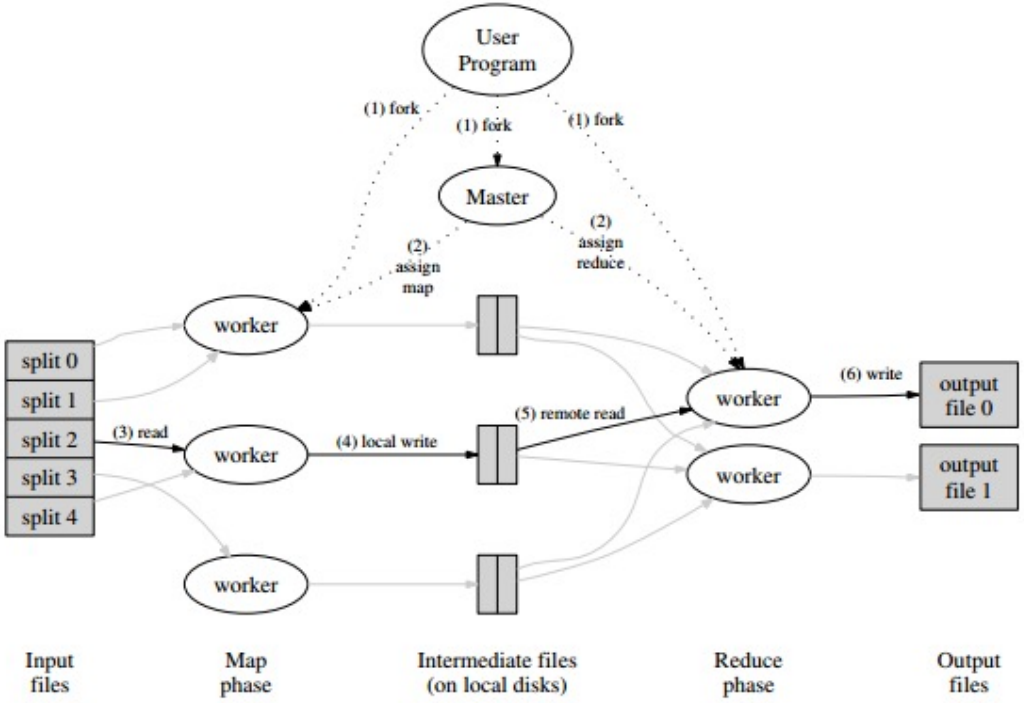



Figure 1: Execution overview

# Fault Tolerance

- **Handled via re-execution**
  - Detect failed workers by periodic hartbeats
  - Re-execute complete (*) and in-progress map tasks
  - Re-execute in progress reduce tasks
  - Task completion committed through master
  - In the paper, they assume the master won't fail

- Robust: report successfully completing jobs where 1,600 out of 1,800 workers have failed
- In Hadoop 3.0 improved by **erasure coding** (**)

(*) Because completed map tasks output is stored on the local disk(s) of the failed machine and is therefore inaccessible. Completed reduce tasks output is accessible (Stored on a global file system)

(**) https://hadoop.apache.org/docs/r3.0.0/hadoop-project-dist/hadoop-hdfs/HDFSErasureCoding.html

# Optimisation: Redundant execution

- Helps when slow workers lengthen completion time
- As the task nears completion, the master can look at the *in-progress tasks*
  - Schedule backup versions of the remaining tasks on idle workers
  - A task is completed when either the main or backup version completes
- Leads to very large efficiencies for low extra overhead

# Takeaway message:

- MapReduce data-level parallelism facilitates automatic control for job execution:
  - Automatic division of jobs into task
  - Automatic placement of computation near data
  - Automatic Load Balancing
  - Automatic recovery from failure (fault-tolerance)
- Users can focus on the problem to be solved